Assessment Overview

Q-Learning for Mastermind

Overview

Implement tabular Q-learning and investigate how the choice of state representation and exploration rate affects performance in Mastermind.

Code Structure

This assignment follows the same pattern used in the class notebooks:

- `MastermindEnv` stores the game state and the rules of the world
- `QTable` is provided separately in `qtable.py`
- `QLearningAgent` stores `self.env`, the learning parameters, and `self.Q`
- evaluation and plotting are written as standalone functions outside the classes

Your Tasks

1. **Complete** the `select_action` and `update` methods in the provided `QLearningAgent` class in `student_template.py`
2. **Find** the optimal history length for the state representation
3. **Investigate** a good exploration rate or regime (epsilon)
4. **Plot** results with proper statistics (mean +/- std from multiple runs)
5. **Write** a report analyzing your results and answering the four analysis questions listed in `student_template.py`

The Game

Mastermind is a code-breaking game:

```
Position 1    Position 2
   [ ]           [ ]
```

- Each position holds one color (colors are numbered `0` to `NUM_COLORS - 1`)
- You guess the secret code
- After each guess, you get feedback:
  - **Black**: correct color in correct position
  - **White**: correct color in wrong position

Example: Secret is `(1, 2)`, you guess `(1, 0)`:

- Position 1: guess=1, secret=1 → Match! Black = 1

- Position 2: guess=0, secret=2 → No match. Is 0 elsewhere? No.
- Feedback: black=1, white=0

State Representation

Each turn record stores 4 numbers:

```
(guess_pos_1, guess_pos_2, black, white)
```

The state is a tuple of the last N turn records (N = `history_length`).

Example game with history_length=4:

Secret is `(1, 2)`.

```
Turn 0: Guess (0,0), feedback black=0, white=0
State = ((0, 0, 0, 0), None, None, None)

Turn 1: Guess (1,1), feedback black=1, white=0
State = ((0, 0, 0, 0), (1, 1, 1, 0), None, None)

Turn 2: Guess (1,2), feedback black=2, white=0  → WIN!
State = ((0, 0, 0, 0), (1, 1, 1, 0), (1, 2, 2, 0), None)
```

Configuration

See the configuration block near the top of `student_template.py` for the initial experiment settings and placeholders used by the scaffold, together with short inline comments explaining what each one controls. Some of those settings are meant to be updated as your earlier experiments inform the later ones.

It is recommended to use `NUM_COLORS = 4` unless runtime is a genuine problem on your machine. If you use `NUM_COLORS = 3`, please state this clearly in your report.

Provided API

Please use the provided `QTable` as given in this assignment. You should not need to modify `environment.py` or `qtable.py`.

From `MastermindEnv`, the main pieces you need are:

- `state = env.reset()`
- `next_state, reward, done, info = env.step(action)`
- `env.num_codes`
- `env.done`

- `env.turn`

The step outputs mean:

- `next_state` : the next state tuple
- `reward` : `10.0` if the current guess solves the code, otherwise `-1.0`
- `done` : whether the episode has finished
- `info` : dictionary with `guess` , `black` , `white` , `turn` , and `won`

Reward scheme:

- positive reward: `10.0` only when the agent wins
- negative reward: `-1.0` on every non-winning step
- this includes the final step of a lost game

From `QTable` , the main pieces you need are:

- `Q.get(state, action)` : returns the current value estimate for that state-action pair
- `Q.set(state, action, value)` : stores a new value estimate for that state-action pair
- `Q.get_max_value(state)` : returns the highest current action value for that state
- `Q.get_best_action(state)` : returns a greedy action; if several actions share the highest value, one of them is chosen at random
- unseen state-action values default to `0.0`

Especially early in training, many actions may therefore have the same Q-value. When that happens, greedy action selection can still move randomly among the tied actions.

Algorithm

The algorithm to implement is tabular Q-learning, as discussed in the module material. The coding task is completed in `student_template.py` .

Experiments

For your experimental comparisons, it is recommended that you use enough independent runs for your qualitative conclusion to look steady. You may find it helpful to start with tens of runs rather than hundreds, but increase the number if your results are still unstable. The exact parameter settings and placeholders are defined in `student_template.py` . When comparing parameter settings, consider both the mean performance and the variability across runs; if several settings have similar means, discuss whether one appears more stable. In many reinforcement learning studies, one would also tune the learning rate (`eta`) and the discount factor (`gamma`). To keep this study short and focused, use the reasonable fixed values provided in the code (`eta = 0.2` , `gamma = 0.99`) while investigating `history_length` and `epsilon` .

A sensible order is:

1. first investigate the state representation (`history_length`)
2. then, using your chosen history setting, investigate exploration (`epsilon`)

3. finally, plot learning curves for your final combined choice

In your report, clearly state the parameter values you used for each experiment. This should include the tested `history_length` values, the tested `epsilon` values or range, the final chosen `history_length` and `epsilon`, and the main run/evaluation settings used to produce your reported tables and plots.

1. History Length

Investigate a small sensible range of `history_length` values. It is recommended that your study include `history_length=1` and only a few larger values to justify your conclusion; there is no need to search large history lengths.

Key questions:

- What happens with history_length=1?
- What history length appears optimal in your study?
- How do the results compare to a random baseline under the same configuration?

2. Exploration Rate (Epsilon)

Investigate a sensible range of `epsilon` values. It is recommended that your range include `epsilon=0.0` and cover a sensible spread within `[0, 1]`, so that you can discuss both low- and high-exploration behaviour. As a guide, a study within an interval such as `[0.0, 0.6]` is usually sufficient; there is no need to search an excessively large range. Start with a coarse sweep using a small number of well-spaced epsilon values. A very fine sweep can be unnecessary and may slow the experiments substantially, so only increase the resolution if needed.

Key questions:

- What happens with epsilon=0?
- In this setup, why might epsilon=0 still give good results? Explain your answer using the properties of the provided setup.
- What value or regime works best?
- How do the results compare to a random baseline under the same configuration?

3. Learning Curves

Plot the mean win rate and mean average turns over training episodes, and include the standard deviation across independent runs so you can assess the variability and reliability of the learning dynamics. Please report how the number of independent runs affected your conclusions, and explain when those conclusions appeared to stabilise.

Evaluation Metric

Average turns uses a penalty for unsolved games: when computing the average, each failed evaluation game is counted as `max_turns + 1` turns.

This captures both success rate AND efficiency in one number.

When reporting results, please include a random baseline computed with the same configuration (especially the same `NUM_COLORS`) as your trained agent.

Files

File	Description
`environment.py`	Mastermind game (provided)
`qtable.py`	Provided Q-table helper
`student_template.py`	Your implementation

Running

```
python student_template.py
```